

Subtipado

En donde repasamos el concepto de subtipo, y revisamos qué implica tener subtipos.

Repaso de tipado



Expresiones

2

1+1

3

"Hola"

atila

elZorro

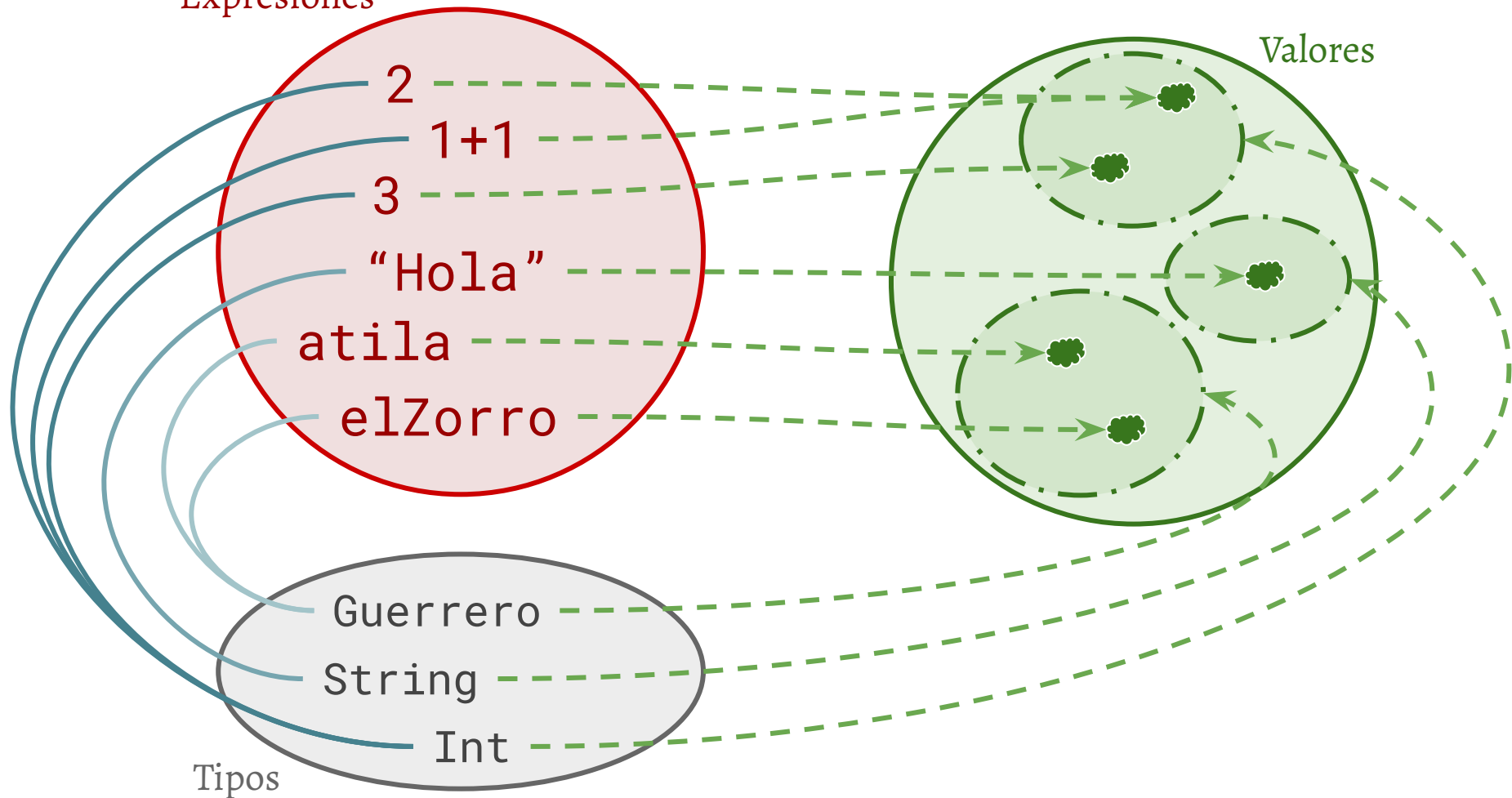
Valores

Guerrero

String

Int

Tipos



Subtipado

The image features a solid red background. A white diagonal line runs from the bottom-left corner towards the top-right corner, creating two distinct triangular regions. The upper triangle is a lighter shade of red, while the lower triangle is a darker shade. The word "Subtipado" is written in a white, serif font in the upper-left area of the lighter red triangle.

Expresiones

mulan

atila

elZorro

Valores



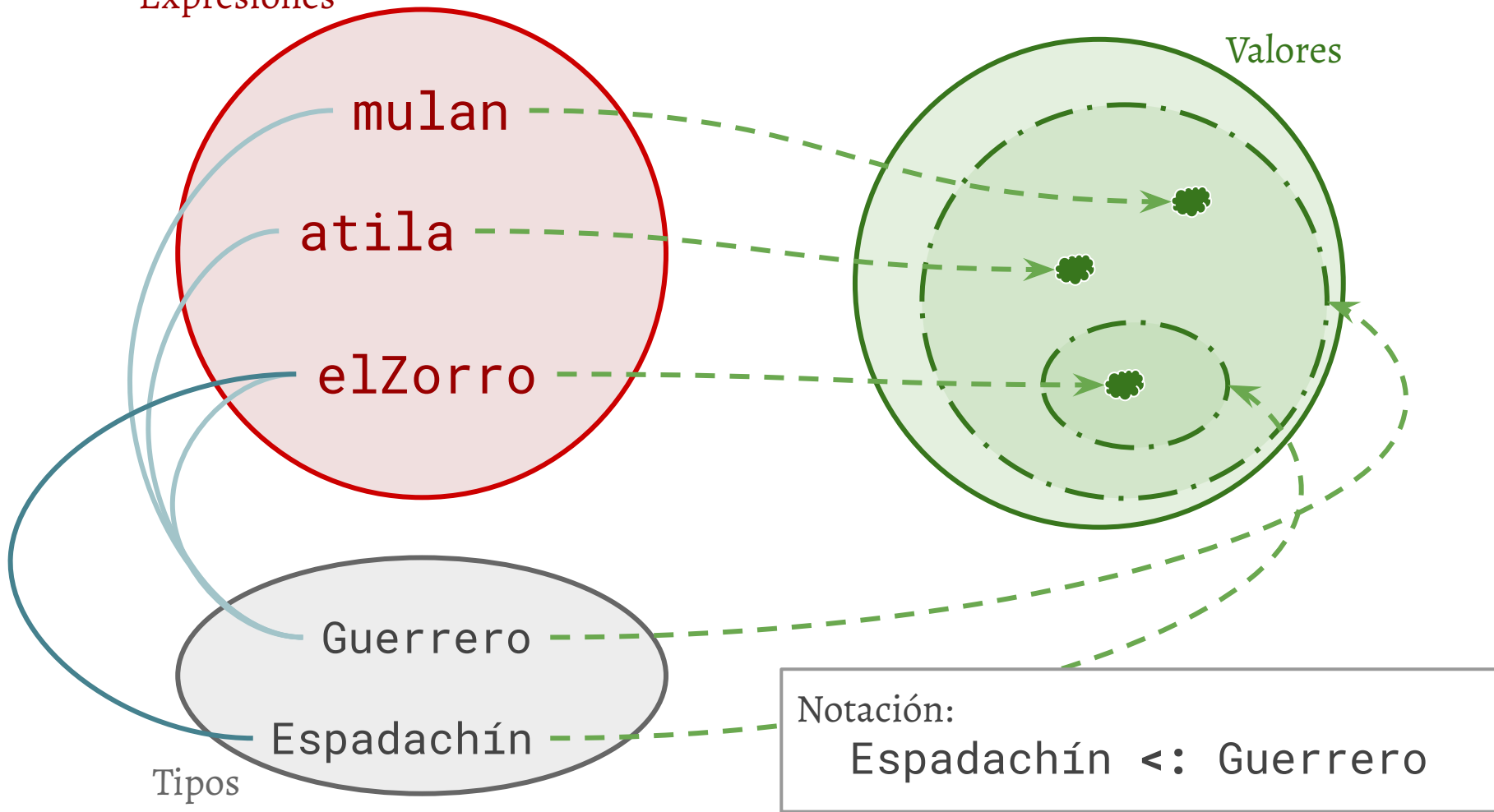
Guerrero

Espadachín

Tipos

Notación:

Espadachín <: Guerrero



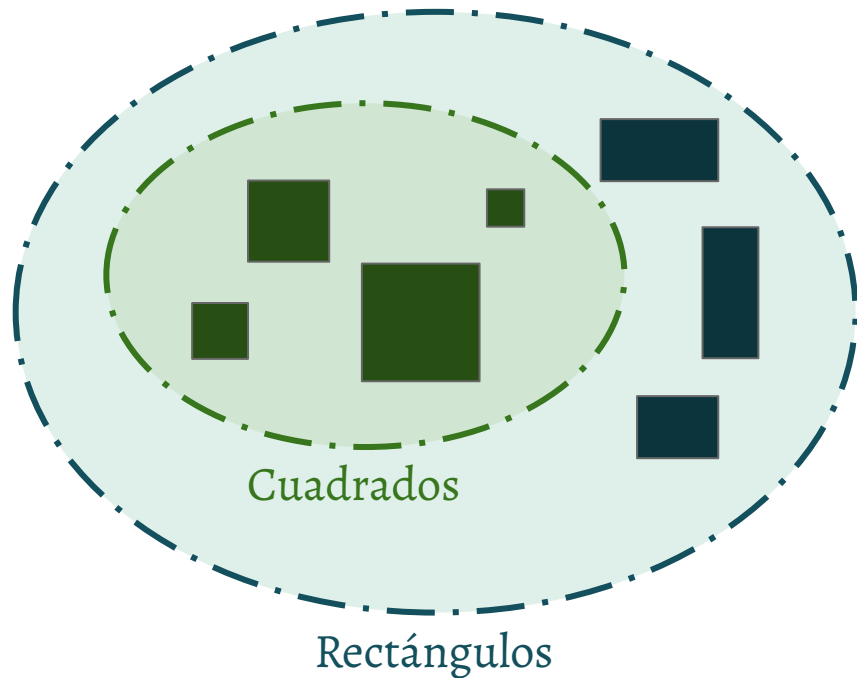
¿Cualquier subconjunto de valores
forma un subtipo?

Ejemplo: Cuadrados y rectángulos

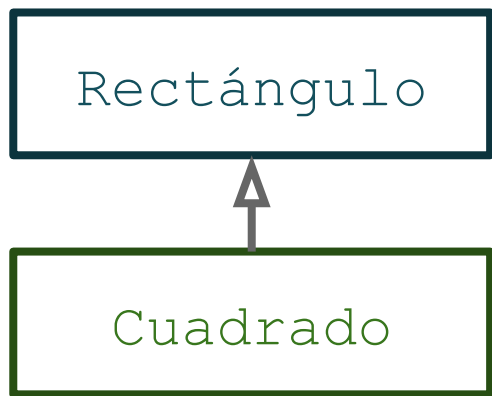
Como parte de nuestro sistema CAD tenemos cuadrados y rectángulos.

Matemáticamente, todo cuadrado es un rectángulo. Es decir, el conjunto de todas las figuras cuadradas es un subconjunto del conjunto de todas las figuras que son rectángulos.

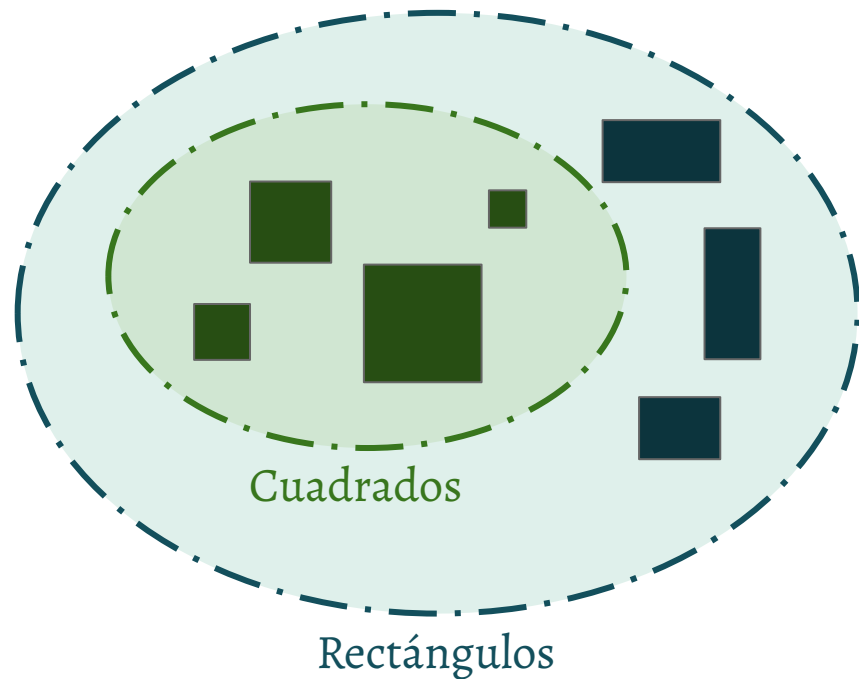
$$\text{Cuadrados} \subset \text{Rectángulos}$$



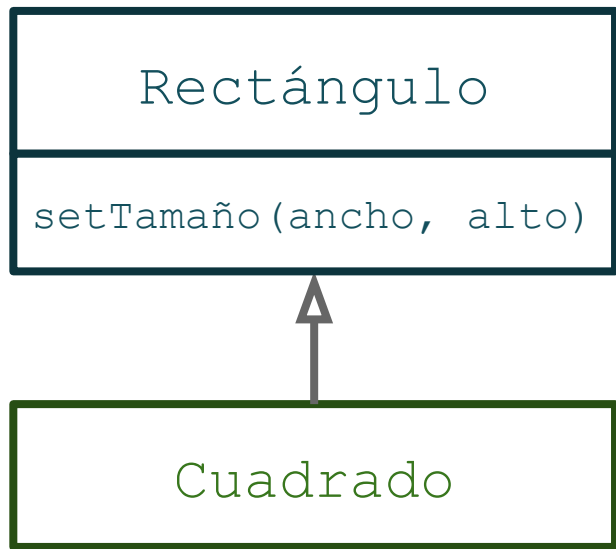
Ejemplo: Cuadrados y rectángulos



“todo cuadrado es un rectángulo”



Ejemplo: Cuadrados y rectángulos

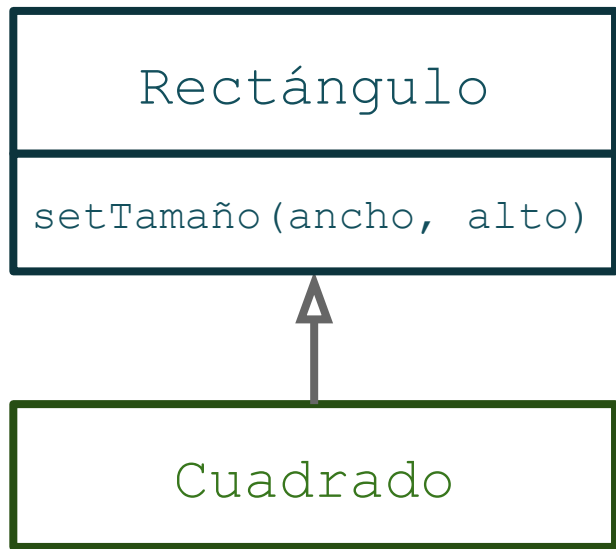


```
def cambiarTamañoDe(unRectangulo: Rectangulo): Unit = {  
    unRectangulo.setTamaño(ancho = 10, alto = 20)  
}
```

```
val unCuadrado: Cuadrado = new Cuadrado(lado = 2)
```

```
cambiarTamañoDe(unCuadrado)
```

Ejemplo: Cuadrados y rectángulos



```
def cambiarTamañoDe(unRectangulo: Rectangulo): Unit = {  
    unRectangulo.setTamaño(ancho = 10, alto = 20)  
}
```

```
val unCuadrado: Cuadrado = new Cuadrado(lado = 2)
```

```
cambiarTamañoDe(unCuadrado) ⚠
```

Herencia

(repaso clase 1)

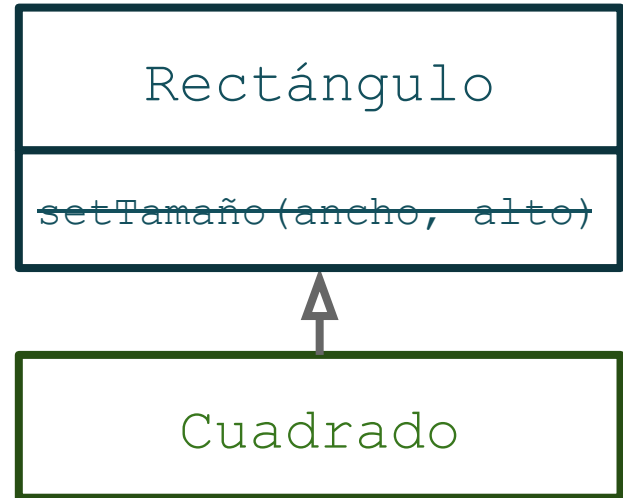
- Me permite reusar comportamiento e interfaz dinámicamente, dado el mecanismo del method lookup (parte mecánica/operacional).
- Representa una teoría del conocimiento de una parte del dominio del problema. Sólo subclasificamos si es válido en el dominio.
- Heurística: “todo *X es un Y*”; **en tanto y en cuanto se comporta como tal, es decir, respeta el mismo protocolo.**

Algunas alternativas

Puede ser una pregunta que le hagamos a los rectángulos, en lugar de tener un subtipo.



Si los rectángulos y cuadrados son inmutables, podría ser que respeten el mismo protocolo.



¿Cualquier subconjunto de valores forma un subtipo?

-  Los valores deben respetar el mismo protocolo/contrato.
- El tipo de un valor no puede cambiar en tiempo de ejecución.
 - Esto implica que su tipo más específico debe conocerse al momento de “crear” o definir el valor por primera vez.
- Hay subconjuntos que el sistema de tipos de Scala no puede representar.

Por ejemplo:

 - Los `Ints` positivos
 - Las listas de 1 elemento, de 2 elementos, de 3 elementos... (tipos dependientes)

El principio de sustitución de Liskov





A Behavioral Notion of Subtyping

BARBARA H. LISKOV

MIT Laboratory for Computer Science
and

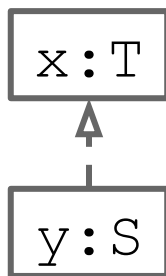
JEANNETTE M. WING

Carnegie Mellon University

¿Qué nos dicen?

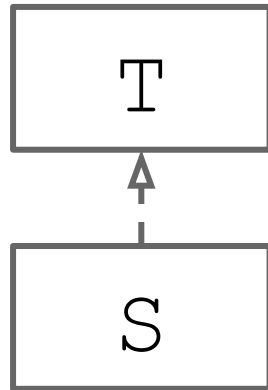
What is needed is a stronger requirement that constrains the behavior of subtypes: properties that can be proved using the specification of an object's presumed type should hold even though the object is actually a member of a subtype of that type:

Subtype Requirement: Let $\phi(x)$ be a property provable about objects x of type T . Then $\phi(y)$ should be true for objects y of type S where S is a subtype of T .



Subtipado => sustituibilidad

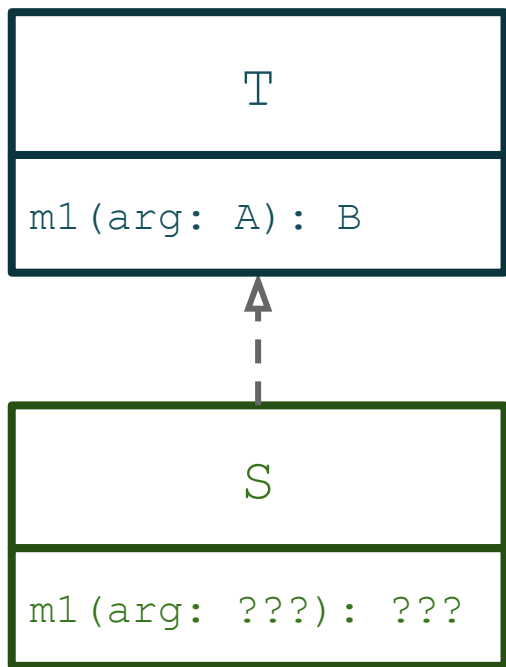
Si S es un subtipo de T, entonces en nuestro programa los objetos de tipo S pueden ser usados en lugar de objetos de tipo T.



Subtipado y contratos



¿Qué relación tienen que cumplir los métodos definidos en subclases* con el contrato de un supertipo?



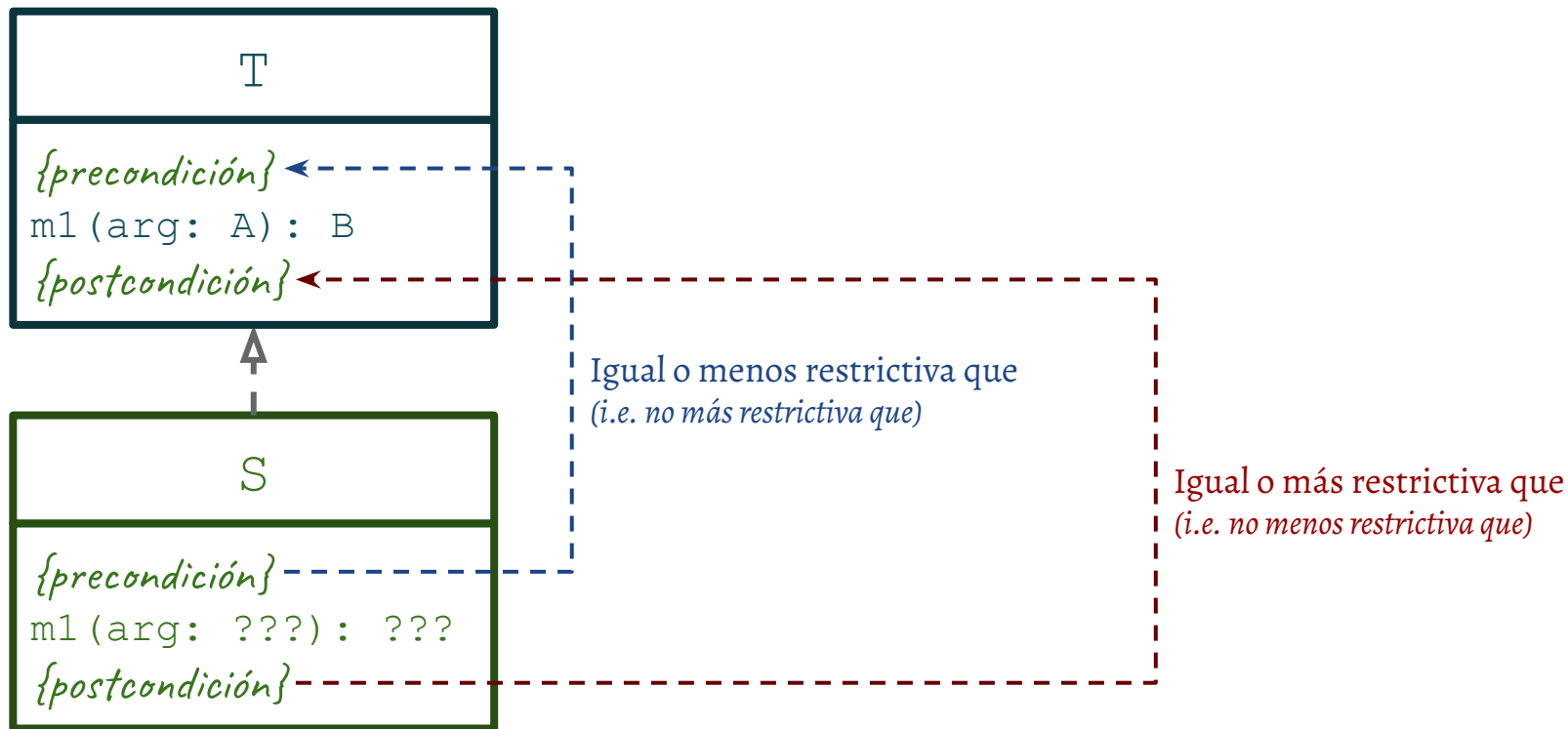
```
val unT: T = ??? ←-----
{precondición}
val resultado: Int = unT.m1(argumento)
{postcondición}
```

Si demuestro que:

“si antes de enviar el mensaje *m1* a un objeto de tipo *T* se cumple *{precondición}*, entonces después se cumplirá *{postcondición}*”

entonces eso debe seguir siendo válido al reemplazar el valor por uno de tipo *S*.

¿Qué relación tienen que cumplir los métodos definidos en subclases* con el contrato de un supertipo?



Repaso: ¿Cualquier subconjunto de valores forma un subtipo?

- Los valores deben respetar el mismo protocolo/contrato.
(principio de sustitución de Liskov)
- El tipo de un valor no puede cambiar en tiempo de ejecución.
- Hay subconjuntos que el sistema de tipos no puede representar.

Por ejemplo:

- Los `Ints` positivos
- Las listas de 1 elemento, de 2 elementos, de 3 elementos... (tipos dependientes)

Varianza



¿Cómo se relaciona todo esto con la varianza?

Supongamos que hemos definido que `Lista[+A]`.

¿Qué restricciones debería cumplir nuestra implementación? ¿Podría haber algún problema si decimos que una `Lista[String]` es a su vez una `Lista[Object]`? ¿En qué casos, y cómo podríamos justificarlo usando lo que vimos?

¿Preguntas?